

Scala Programming Practice Questions

Basic to Intermediate Level Exercises

Basic Level

1. Variables and Types

Write a program that declares a `val` for your name and a `var` for your age, then prints "Hi, I'm X and I'm Y years old." Try reassigning the `val` and observe the compiler error.

2. String Manipulation

Given a string "Scala is fun", write functions to: reverse it, count vowels, and check if it's a palindrome (test with "madam").

3. FizzBuzz

Print numbers 1 to 100; for multiples of 3 print "Fizz", for 5 print "Buzz", for both print "FizzBuzz". Do it with a `for` loop, then rewrite using `(1 to 100).foreach`.

4. Basic Pattern Matching

Write a function `dayType(day: String): String` that returns "Weekend" for Saturday/Sunday and "Weekday" otherwise, using `match`.

5. Option Handling

Write a function `safeDivide(a: Int, b: Int): Option[Int]` that returns `None` on division by zero instead of throwing. Use it in a `for-comprehension` over two divisions.

6. Collections Basics

Given `val nums = List(4, 8, 15, 16, 23, 42)`, compute: sum, max, the even numbers only, and each number doubled — using `map/filter/fold` (not manual loops).

7. Case Classes

Define a case class `Person(name: String, age: Int)`. Create a `List[Person]`, then find everyone over 18 and print their names.

Intermediate Level

8. Higher-Order Functions

Write a function `applyTwice(f: Int => Int, x: Int): Int` that applies `f` to `x` twice. Test it with a function that squares a number.

9. Custom Sorting

Given List(("Alice", 25), ("Bob", 19), ("Carol", 30)), sort by age ascending, then by name descending, using sortBy and Ordering.

10. Recursion + Tail Recursion

Write a recursive factorial function, then rewrite it as tail-recursive using @tailrec and an accumulator. Explain why the first version could stack-overflow on large inputs and the second can't.

11. Traits and Mixins

Define a trait Animal { def sound: String }, then create Dog and Cat classes that implement it. Write a function that takes List[Animal] and prints each one's sound.

12. Pattern Matching on Case Classes

Model a simple shape hierarchy: sealed trait Shape, with Circle(radius: Double), Rectangle(w: Double, h: Double), Triangle(base: Double, height: Double). Write an area(shape: Shape): Double using pattern matching. (Sealed + match gives you exhaustiveness checking — try removing a case and see the warning.)

13. Working with Maps

Given a List[String] of words, build a Map[String, Int] counting the frequency of each word (e.g., word count on a sentence), using groupBy and mapValues/view.mapValues.

14. Custom Exceptions + Try

Write a function that parses a string to an Int and wraps it in Try. Handle both Success and Failure cases, printing a friendly error message on failure instead of letting the exception propagate.

15. Implicit Parameters (basic)

Write a function greet(name: String)(implicit greeting: String): String. Define an implicit val for the greeting and call the function without passing it explicitly.

16. For-Comprehensions with Collections

Given two lists, val a = List(1,2,3) and val b = List("x","y"), use a for-comprehension to generate all pairs (Int, String) — essentially a manual cartesian product.

17. Mini Project — Bank Account

Build a case class Account(id: String, balance: Double) with pure functions deposit(acc, amt): Account and withdraw(acc, amt): Either[String, Account] (returning an error message on insufficient funds instead of mutating state). This is a good intro to immutability + Either for error handling.

18. Type Classes (light intro)

Define a trait Show[A] { def show(a: A): String }, provide instances for Int and Person, and write a generic printAll[A](xs: List[A])(implicit s: Show[A]) function. This previews idiomatic Scala type-class patterns used heavily in libraries like Cats.

Tip: Work through these in order — each intermediate question builds on concepts (immutability, pattern matching, functional collections) introduced in the basic set.